

Classical vs. End-to-End Learning Approaches to Robot Pathfinding in Digital Twin Environments

1st Miguel Gapud

School of Computing
University of South Alabama
Mobile, Alabama

mvg2021@jagmail.southalabama.edu

2nd George W. Clark

School of Computing
University of South Alabama
Mobile, Alabama

georgewclark@southalabama.edu

3rd J. Todd McDonald

School of Computing
University of South Alabama
Mobile, Alabama

jtmcdonald@southalabama.edu

4th Na. Gong

College of Engineering
University of Alabama
Tuscaloosa, Alabama

nagong@ua.edu

Abstract—Pathfinding is essential for autonomous robots. Graph-based classical and machine learning approaches are often compared separately. This paper compares end-to-end pathfinding with Proximal Policy Optimization and an Alexnet architecture against Hybrid A*. A digital twin in Unity3D with the Clearpath Dingo robot is used for testing. PPO controls the robot using ROS-Noetic with a camera as its sensor. Hybrid A* and its controls are implemented in Unity3D. Path length, traversal time, training time, path calculation time, and storage space are evaluated. PPO had a 28.68% success rate over 10,801 test runs. In successful runs, PPO produced paths averaging 13.91 meters with a standard deviation of ± 0.22 meters, traversed in 34.32 seconds with a standard deviation of ± 4.54 seconds. Hybrid A* produced a path averaging 13.36 meters, traversed in 22.69 seconds.

Index Terms—Digital twin, end-to-end machine learning, simulation modeling, pathfinding, robotics.

I. INTRODUCTION

Autonomous mobile robots are increasingly deployed across industries such as warehousing, logistics, and healthcare, where they navigate spaces without direct human control. Pathfinding, or path planning, enables these robots to move between locations while minimizing distance, risk of collision, or energy use [1].

Classical algorithms like A*, D* Lite, and Hybrid A* provide deterministic solutions based on graph search and heuristic optimization. In contrast, reinforcement learning (RL) methods such as Proximal Policy Optimization (PPO) learn policies directly from sensory input, enabling end-to-end navigation without explicit mapping. While these approaches are often studied separately, direct comparisons in common environments remain limited.

This paper compares end-to-end PPO pathfinding using an AlexNet-based architecture against Hybrid A*. A Unity3D digital twin of a Clearpath Dingo robot is used for evaluation. Metrics include path length, traversal time, training time, path calculation time, storage cost, and success rate. The case study highlights trade-offs between learned and classical methods, offering insight into their relative strengths for simulation-based robotic navigation.

II. BACKGROUND

A. Machine Learning Pathfinding

Machine learning (ML) algorithms learn to generalize from input data to perform tasks such as pathfinding. Q-learning develops policies from state-action rewards and stores them in a Q-table. Kim and Lee [9] applied Q-learning to a simple 25-cell grid with cardinal movement.

Mnih et al. [13] developed a versatile ML method called a Deep Q-Network (DQN) with applications in various domains, including pathfinding [6]. Zhao, Qu, and Xu [22] demonstrated DQN in a dedicated pathfinding algorithm in the Simulation of Urban Mobility platform (SUMO). They found that in complex situations like active multi-lane roads, the many possible steering and acceleration controls create a cumbersome Q-table to search in real-time. To address this, they created a reward function based on DDQN and trained a neural network to approximate Q values. Their RL strategy performed well in environments with dynamic obstacles.

Zhao et al. [23] showed a similar way to use ML for pathfinding in mobile robots. Their approach implemented ML for local pathfinding, specifically for autonomous vehicles to safely choose paths moment-to-moment on a road. Using Q-learning, a simulated vehicle moved smoothly when changing lanes and kept a safe distance from cars in front and behind it, based on its kinematic model and surrounding vehicles.

PPO is a widely used RL algorithm for tasks such as pathfinding [19]. PPO learns a stochastic policy and iteratively improves it by exploring nearby actions and updating based on observed rewards, using gradient ascent to maximize advantage.

B. Classical Pathfinding

Classical algorithms use deterministic mathematical search. A* and D* Lite remain influential: A* expands from the current location, while D* Lite searches backward from the goal [7, 20]. Liauw et al. [11] found D* Lite faster and more effective in dynamic environments, though A* performs better in small static spaces.

Hybrid A*, proposed by Dolgov et al. [3], extends A* by incorporating vehicle kinematics to generate smooth, drivable paths. Unlike A* and D* Lite, Hybrid A* accounts for

continuous positions within nodes and uses Voronoi fields to discourage paths near obstacles. Although not always optimal, Hybrid A* remains practical for non-holonomic robots.

C. Related Work

Zhang et al. [21] enhanced PPO-based path planning by integrating an Intrinsic Curiosity Module (ICM) and Long Short-Term Memory (LSTM) into an Actor-Critic framework, termed LI-PPO. They compared it against PPO variants and Deep Deterministic Policy Gradient, using reward functions based on goal-reaching, orientation, distance reduction, and collision penalties. Experiments in simulation and on a TurtleBot3 Burger showed LI-PPO outperformed all other tested algorithms in convergence rate, success rate, and average reward.

Gao et al. [5] introduced PRM+TD3, combining a Probabilistic Roadmap (PRM) with Twin Delayed Deep Deterministic Policy Gradient for local navigation. They compared it with improved A* algorithms, including A*-DWA and A*-TEB. PRM accelerated TD3 training by providing a global roadmap, while TD3 handled local decisions. In 2D and 3D Gazebo environments, TD3 outperformed DDPG and PPO during training, though PRM+TD3 produced longer paths and traversal times than A* based methods in large spaces.

Their reward function used a piecewise design, rewarding goal achievement, penalizing wall proximity, and adjusting based on distance. TD3 training progressed incrementally from 2D Gym to larger 3D Gazebo maps to improve convergence. Using a TurtleBot3 with lidar, PRM+TD3 ultimately achieved a 100% success rate, but still lagged A* based methods in efficiency.

III. METHODOLOGY

To investigate how machine learning and classical approaches to pathfinding compare to one another in the context of a simulation environment, a simulation environment is created, and both a machine learning approach, E2E PPO, and a classical approach, Hybrid A*, are implemented. In these implementations, PPO receives images from a camera mounted on the front of the robot as input and sends control commands as output. Hybrid A* has global knowledge of the environment in a grid-based graph representation.

In the simulation environment, both pathfinding algorithms are evaluated on their ability to navigate a static path marked by traffic cones set along the desired path, starting from a set point. After PPO has sufficiently trained on this path, 10,000 runs are conducted with the trained model to collect metrics. The metrics from the successful runs from those 10,000 are compared against a single run of Hybrid A*, because Hybrid A* is a deterministic algorithm for which there would be no meaningful information gathered from multiple samples.

A. Metrics

The relevant metrics for answering this question are as follows:

- Training time - The length of time in hours between the beginning of training to a trained model. This data is gathered by the training script getting the system time at the starting step and the system time at every 100 epochs.
- Path length - The distance in meters traveled by the robot between start and goal points. It is calculated by appending the robot's center coordinate to an array of points at the start of the first movement and at every subsequent update until the goal is reached. The distance between consecutive points in the array is summed to approximate the total distance traveled.
- Path calculation time - Time taken in seconds between beginning of calculation and first movement. A timestamp is recorded when path calculation begins and when it ends. The difference between these times is the path calculation time.
- Storage space - memory required in MB needed for the approach to be implemented. For the Hybrid A* approach, the storage needed by the compiled algorithm during runtime is considered as well as any graphs created and stored by the algorithm to calculate the path.
- Traversal time - The time in seconds taken for the robot to navigate between the start to goal points. As soon as the first movement is taken, a timestamp is taken. When the robot reaches its final waypoint, a second timestamp is recorded. The time taken is the difference between these two timestamps.
- Success rate - Percentage of successful runs. This is calculated for PPO by dividing the number of times the robot reached its goal by the total number of runs. Hybrid A* is a deterministic method, so this is not a consideration for that algorithm in a static environment.

To directly evaluate the pathfinding of the PPO and Hybrid A* algorithms, the path produced by the Hybrid A* algorithm is tested against many paths produced by the trained PPO algorithm. There does not seem to be a single set of metrics used in pathfinding research, as direct comparison with other established methods seem to be the most valuable standard for understanding the quality of pathfinding algorithms [16]. However, these metrics seem to have been used in the literature before for comparing machine learning and classical pathfinding approaches.

B. Data Analysis

For the machine learning method, training time is the uniquely relevant feature, as classical methods do not need time to train. For the classical method, path calculation time is the uniquely relevant feature, as the end-to-end machine learning method quickly calculates the path while the robot is in motion, avoiding the pitfalls of the front-loaded path calculation of classical methods. The remaining metrics are relevant for both approaches. To compare these different methods, the paths they navigate are graphed in terms of time taken and path length. Because classical methods have deterministic performance, its path is graphed as a single point against the distribution of paths taken by the machine learning

method. The relevant feature of this graph is how consistently well the machine learning method performs compared to the classical method for any metric.

C. Environment Setup and Unity3D

The simulation environment is made with a combination of Unity3D 2022.3.20f1 and ROS-Noetic, running on WSL2. Unity3D add-ons were used to make this project possible. The URDF importer add-on allows Unity3D to import files in the Universal Robot Description Format, which is a standard for specifying robot configurations in ROS. In Unity3D, this defines all of the joints of the robot as Articulation Body components as well as all the necessary collision. This allows the robot in simulation to be easily and accurately replicable. The robot used is the Clearpath Dingo URDF, imported directly from the Clearpath website. A 3D model of the IntelRealsense D435 Multibody camera is placed to scale as child gameObject of the Dingo's chassis_link – the first child of the imported URDF that contains an articulation body. The camera gameObject which collects the visual data is placed as a child of its gameObject in the location of the lens of the camera on the depth camera 3D model, facing forward. The camera component is edited to match the specifications of one of the cameras of the IntelRealsense D435 Multibody. However, due to misunderstandings of the script simulating the camera, the field of view was slightly wrong. This likely would affect the transferability of this model into the physical robot but should not be relevant for this application. To interface with the Dingo, the ROS-TCP-Connector package enables Unity3D to send and receive ROS messages.

The environment is a digital twin 3D scan of a hallway and some rooms connected to it created using Polycam. Polycam is an application for mobile devices that can create 3D models of spaces by scanning spaces using the device's built-in LiDAR and exporting them as a GLB file. To import these files into Unity3D, the package GLTFUtility is used to convert the GLB into texture files applied on a mesh file. To guide the robot, waypoints are used. These waypoint prefabs contain two child objects: a sphere, and a small traffic cone. The spheres are invisible to the robot and primarily function to guide the reward function of the robot. They have trigger colliders and are placed on a layer invisible to the Dingo's camera because they do not exist in the real world. The scale of these spheres is (1, 1, 1.5), and they are centered on the middle of each traffic cone's base. In contrast, the traffic cones are visible to the Dingo's camera, and the Dingo can collide with them as though they were a wall.

Custom scripts are created to provide functionality to the simulation environment. The movement script for the Dingo moves the robot based on received ROS Twist messages, and it sends back Float message rewards based on the reward function in the Unity scene. This script is based on the controller package used by the Dingo, with modifications to more accurately mimic the real-world movement of the Dingo when moving it using a controller. Because this script deals with the reward function, it also tracks whether the Dingo has

hit a wall. The script is a component of the chassis_link object of the built Dingo. The camera script was created by Platt and Ricks [15]. It collects information from the Dingo's camera object and sends that data to ROS as an RGB8 ROS image message. This script is configured as a component of the root object of the Dingo, and it finds the camera component from the children of the Dingo object. For machine learning, the script is edited to truncate the image to be 227x227 pixels.

A Waypointing script is created which manages the waypoint spheres. This script exists as a component of each sphere object of the waypoint prefab. When the robot enters a sphere's trigger collider, the sphere communicates to the movement script that the waypoint has been reached, causing the sphere to disappear and reward function to update. For the purpose of training multiple batches of the simulation at once, a scene manager object and script is used to instantiate any number of copies of the rest of the environment offset from each other along the y-axis so they cannot see each other, including the digital twin, robot, and waypoints using a single ROS-Integer command denoting the number to instantiate. It is also able to reset all environments using a single boolean command from the ROS script, uncollecting all waypoints, resetting all robots to their original positions, and activating the reset scripts in any data collection scripts to prepare for the new run.

Upon creating the environments, the scene manager script renames the objects to be called "env_" followed by the number of instances currently created. In sum, each environment creates 3 unique ROS topics: its robot control topic (subscriber), reward topic (publisher), and camera topic (publisher). The scene manager adds 2 more topics, subscribing to the reset topic and instantiation topic. To avoid messages colliding, each environment uses its own set of ROS topics, with names that have the string "env_" followed by its environment number prepended to the topic names.

D. Robot Operating System (ROS)

The version of ROS used is ROS-Noetic on Ubuntu 20.04 for WSL2. WSL2 is used rather than a virtual machine or a separate computer to simplify networking and facilitate the use of multiple Linux environments running in parallel. The ROS-TCP-Endpoint repository from the Unity Robotics GitHub [17] was cloned locally as a workspace. All ROS-side custom scripts were created in the scripts section of the cloned ROS-TCP-Endpoint source file. These include a testing script, a training script, the PPO class, and the neural network class. The machine learning algorithm is PPO, and it and all of its components aside from the reward function are implemented in the ROS workspace.

E. PPO Algorithm

The machine learning algorithm, PPO, runs on the ROS-Noetic virtual machine, and its reward function runs in the Unity3D environment, sending the calculated rewards to the PPO algorithm via ROS messages. The classical pathfinding algorithm, Hybrid A* runs entirely in a Unity3D environment.

An end-to-end, Actor-Critic PPO approach is used, written using Python's PyTorch machine learning framework in combination with rospy. This implementation of the PPO algorithm using PyTorch was based on the repository created by Barhate [2] but heavily modified to interface with ROS. Actor-Critic PPO uses two neural networks, an actor who makes the actions and a critic who makes evaluations of actions. Both neural networks are convolutional neural networks based on the architecture of the Alexnet image classifier by Krizhevsky, Sutskever, and Hinton [10] but the input is an RGB8 color image with the dimensions of 227x227 pixels, and the actor neural network's output layer produces a tensor containing two floats rather than a classification, while the critic neural network's output layer is a single float value. The actor's two floats correspond to controls for the Dingo's differential drive wheels. During training, the reward values created by a custom movement script in Unity3D are sent as the rewards for PPO. The architecture is shown in figure 1.

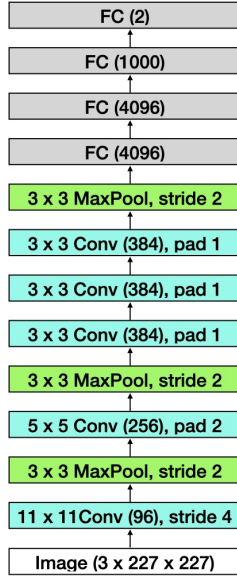


Fig. 1. Alexnet Neural Network Architecture

The PPO algorithm is trained using a batch size of 5. 120 steps are taken per epoch, and updates occur after each epoch. In Unity3D, the timescale is set to x5, with a maximum allowed timestep of 0.03333334 simulation seconds and Fixed Timestep of 0.01 simulation seconds. Training is conducted until loss does not improve significantly anymore as PPO converges. When all batches reach a waypoint, after hitting a wall, they will respawn randomly in a ring centered on that checkpoint. Randomly, they will instead respawn at the starting position instead of the most recent checkpoint, to not lose the ability to navigate from the beginning. Every 100 epochs, the relevant metrics and a copy of the neural network's weights are saved.

F. Hyperparameters and Reward Function

The hyperparameters were chosen through trial and error. For the convolutional layers of the neural network, the

LeakyReLU activation function is used. The activation function that leads into the final layer is the Sigmoid function, and the activation function for the final layer is Tanh. The learning rate of the actor was 0.00001, and the learning rate of the critic was 0.00005. The number of epochs was 49500, and the GAE algorithm by Schulman et al. [18] was used to calculate the relative advantages for each update. During training, rather than decaying the action standard, it is set to 0.8 for the duration of training. The reward function is as follows:

When not contacting a wall:

$$r(s, a)_t = \left(\theta_t + \theta_t \cdot \left(\frac{k_d(d_i - d_t)}{d_i} + x_t \right) + k_n n - k_s s \right)$$

- $r(s, a)_t$ – Reward of the combination of state s and action a at time step t
- t - Current time step. Number of times a movement command is executed and corresponding reward is calculated.
- θ_t - Angle between a unit vector from the center of the Dingo to the direction it is facing and a vector starting from the center of the Dingo's chassis and ending at the nearest waypoint at time step t , normalized so an angle of 0 results in $\theta = 1$ and an angle of 180 results in $\theta = -1$, so θ is a uniform value from $[-1, 1]$
- d_i - Initial distance between the center of the Dingo's chassis and the nearest waypoint, set at the moment that a given waypoint becomes the nearest waypoint.
- d_t - Distance between the center of the Dingo's chassis and the nearest waypoint at time step t , updated at every time step.
- x_t - Forward component of the movement command sent by the PPO algorithm.
- k_d - Constant scalar for the distance term of the reward function. Set to 5.
- n - Number of waypoints reached this epoch. This metric is added so the robot does not think that it can get higher reward for staying close to a waypoint rather than simply collecting it.
- k_n - Constant scalar for the amount of reward per waypoint hit. A value of 7 is used.
- s - The number of simulation steps the agent has spent without resetting or reaching a waypoint.
- k_s - Constant scalar for the amount of punishment per timestep taken. A value of 0.125 is used.

When contacting a wall or after 25 simulation seconds without touching a waypoint:

$$r(s, a)_t = -35$$

After reaching the final waypoint:

$$r(s, a)_t = 60$$

During training, after reaching a reward of either 60 or -35, the Dingo is reset to its initial position. After all robots reach a waypoint during a single epoch, the reset point for each batch

is set to a random point in a 1m diameter circle centered about the waypoint that was reached. At random, the reset point will be changed to the initial position rather than the furthest waypoint reached so the neural network does not lose its ability to navigate the earlier section of the path quickly.

G. Hybrid A*

The Hybrid A* algorithm is used to produce a path in the same environment as PPO was trained in, then the metrics are compared and evaluated. To set up the Hybrid A* project in Unity, a proprietary implementation of Hybrid A* in Unity by Nordeus [14] was used. To function, Hybrid A* needs to quantize the space into grid squares. By default, Unity uses a base unit of 1 unit = 1 meter. The grid creation in this implementation of Hybrid A* uses integer-sized unit squares, so before quantizing, the digital twin and the Dingo's sizes were scaled up by a factor of 10. This was done to properly capture the rough edges of the digital twin. This scaling did not have any effects on the pathfinding or kinematics, as the weights of objects and relative distances are held constant after scaling. A script was then developed to quantize the scan of the digital twin into a 2-dimensional grid equivalent using squares of 0.25 units to mark all obstacles and walls. These squares were added to the Hybrid A* grid to be considered as obstacles. When reporting distances in the results section, the actual measured value is divided by 10 to scale to the size of the actual environment.

Once the digital twin was converted into these squares, the kinematics of the robot were set. The max speed of the Dingo and all other measurements were scaled by 10 to accurately reflect the structure of the scaled-up space when the Hybrid A* algorithm creates paths, though weight measurements were unchanged. The destination set for testing was set to be slightly behind the final waypoint, at (75.5, 0, 21), with a rotation of -90 degrees about the y axis, This is to allow the Dingo to be driving at its full speed as it intersects its final waypoint, matching how the PPO algorithm learns to navigate at as high a speed as possible without slowing down when reaching its destination.

H. Known Limitations

The E2E PPO algorithm as implemented does not develop any form of mapping or generalization of the inputs it receives. If the reward function changes or if the waypoint locations changed, the algorithm would likely need to relearn its interpretation of all images in terms of expected reward and action. This implementation of PPO would not generalize well to any form of validation tasks or environments.

The implementation of Hybrid A* is not implemented to the exact specifications as in the paper that introduced it. Dolgov et al. [3] defined Hybrid A* to use different node sizes depending on the density of obstacles, increasing the efficiency of the map calculation. The mapping in this implementation takes excess time as a uniformly high precision, causing excess precision to be used than needed on open spaces. This should not affect

TABLE I
COMPARISON OF PPO AND HYBRID A*

Metric	PPO	Hybrid A*
Path Length (m)	13.91 $\pm$ 0.22	13.36
Traversal Time (s)	34.31 $\pm$ 4.54	22.69
Success Rate	28.68%	100%
Storage Size	475 MB	< 1 MB
Calculation Time (s)	N/A	6.93
Training Time	161.7 h	N/A

the pathfinding ability of Hybrid A*, but it will cause the path calculation time to be longer than a correct implementation.

IV. RESULTS

A. Analysis

The comparison results are shown in table I. The average difference between the path lengths of the two approaches was 0.55m. The traversal time for paths of the PPO algorithm was longer than that of Hybrid A* by 11.63 seconds on average. Out of the successful runs, the trained PPO algorithm created a path shorter than that of the Hybrid A* algorithm in only 14 runs and traversed its path in a lower time than Hybrid A* in 2 runs. Conducting a one-tailed z-test on the sample of successful runs of the PPO algorithm with a path length shorter than that of Hybrid A* resulted in a p-value of 1, and a one-tailed z-test on the runs of the PPO algorithm with a traversal time lower than that of Hybrid A* also resulted in a p-value of 1. As these p-values are much larger than .05, the probabilities of these occurrences were statistically insignificant. This implementation of PPO is expected to perform worse than Hybrid A* in both of those metrics. Figure 4 compares the distribution of PPO path lengths with Hybrid A*.

The paths generated by the PPO and Hybrid A* algorithms were visually similar, with the main qualitative difference being that the path generated by the Hybrid A* algorithm is closer to the walls during its right turns. See figures 2 and 3. A quantitative difference in the pathfinding of the two algorithms not captured in the chosen metrics is that the PPO algorithm would sometimes direct the robot to stop and turn in place after exiting the first door, taking advantage of the Dingo's differential drive to rotate without moving positionally. A metric such as average velocity as a function of time may have been able to capture this difference. The PPO algorithm created 3098 successful runs out of 10801, for its successful path traversal rate of 28.68%. Figure 5 shows the distribution of path traversal times generated by PPO as compared to the Hybrid A* algorithm.

For Hybrid A*, the map took 2.80 seconds to be calculated, and the path took 4.13 seconds to be calculated, for a total of 6.93 seconds of calculation time. The implementation of the PPO algorithm does not have a path calculation time because it does not calculate a global path, rather it computes the next

most appealing option from the image input while the robot is still carrying out the previous movement command. However, even considering the map and path calculation time as time elapsed for the Hybrid A* path traversal time, Hybrid A* has a traversal time of 29.62 seconds, which is still 4.69 seconds less than the traversal time of the path navigation of the PPO algorithm on average.

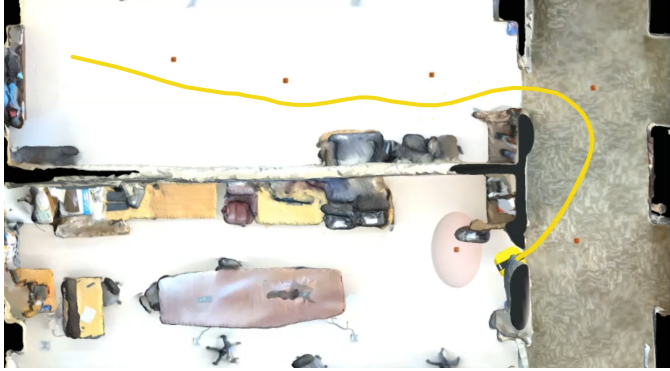


Fig. 2. Example successful path (PPO algorithm)

Training time is a large investment that needs to be taken for any machine learning algorithm. The long training time must be offset by the quality of performance to be justified. The training time of the PPO algorithm was 161.7 hours, which is just short of 1 week.

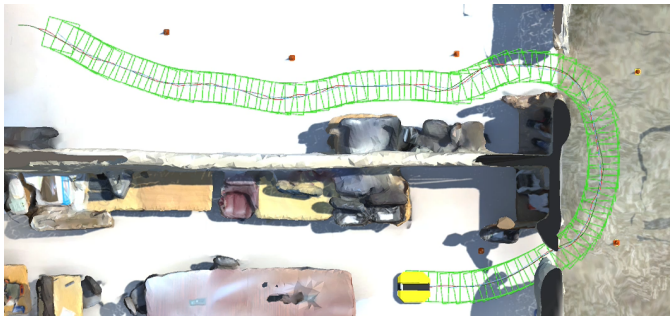


Fig. 3. Path produced by the Hybrid A* algorithm

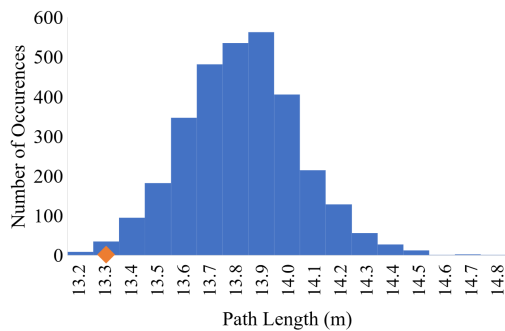


Fig. 4. Distribution of path lengths generated by PPO. The diamond represents the path length generated by Hybrid A*.

B. Limitations

This implementation of PPO was suboptimal. Although it only achieved a 28.68% completion rate, this was not always due to crashing into walls. It may also be that with its current position, it would not be able to fit into the doorway and proceed down the hallway as shown in Figure 6. Sometimes it would fail because it crashed into a wall as seen in Figure 7. This is likely due to its reward function not being properly aligned with the intended goal. During training, the robot would initially attempt to complete the intended path, but at epochs above 49500, it slowly learned to not attempt a path through the second doorway to reach the final waypoint. Instead, it learned to drive past the room and down the hallway, then later it learned to stop just in front of that doorway until its time ran out, likely to maintain the orientation and proximity terms of its reward function without risking the collision punishment. Because the PPO algorithm was able to learn to navigate the space without colliding with obstacles, this issue was likely not in the machine learning code or neural network. It was likely due to the reward function as defined being misaligned to the actual goal of traversing the defined path completely. With the appropriate changes to the reward function, this could have been rectified. But time constraints did not allow for retraining with a new reward function to verify. Many candidate functions would likely need to be trained on for many hours for the differences for a viable reward function to be found.

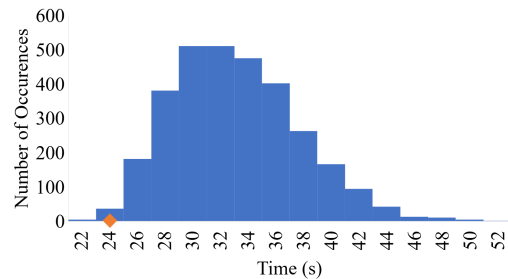


Fig. 5. Distribution of traversal times generated by PPO. The diamond represents the traversal time of Hybrid A*.

The pipeline of training a machine learning algorithm using a ROS-Unity3D connection is rather unconventional. A primary advantage of simulated training over real-world training is that training can be batched, using multiple environments running in parallel. Because of this in combination with the ROS connection used, this problem becomes a networking, multiprocessing, real-time, and machine learning problem. On their own, each domain has its own issues that can make them difficult to debug. When combined, the difficulty in debugging and solving problems is compounded. Because of this and problems that caused time losses that will be elaborated on, the PPO algorithm was never trained to a point where it had deterministically high success in navigating its path.

C. Issues

Other issues encountered had to do with latency demands and issues associated with parallelizing an implementation not initially designed to handle parallelizing, ensuring that each environment was sending and receiving the correct information. One of the biggest issues with this configuration is that there are no error codes when something goes wrong in the machine learning. The code fails silently, so many print statements and try except code is needed.

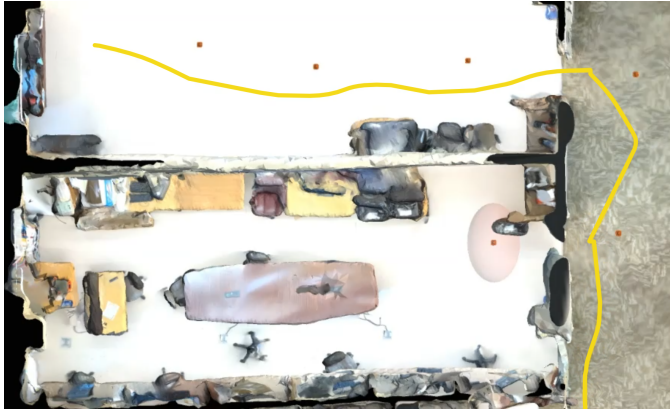


Fig. 6. Example failed long path (PPO algorithm)

Many of the most difficult issues that came up had to do with the intersections of these domains. For example, when accepting image messages in ROS, initially, the PPO training algorithm was reading from the first message sent and proceeding to iterate through each image sent in order. This is an issue because the camera sent images faster than the ROS script and machine learning could process them. The script to use the next image in a queue rather than using the most up-to-date image, rather than the desired behavior of ignoring all of the images sent in the meantime. This bug was unintuitive to discover because it was not a bug with clear feedback; it was an implementation fault written because of how the training algorithm was subscribed to the image topic.

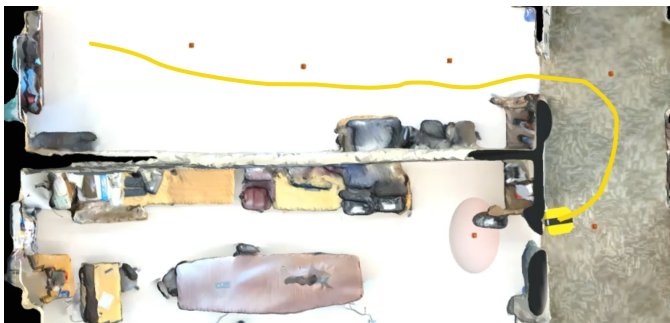


Fig. 7. Example failed short path (PPO algorithm)

The subscriber used queue size of zero, which should theoretically discard the oldest images, but it did not fix this issue. The issue could only be noticed because the algorithm did not seem to be improving. The issue could not be caught

by testing the topic using a simple camera subscriber program showing the images being published over the ROS topic; it had to be found by outputting the images currently being processed by the machine learning program as the training program ran. A possible solution is to only send images from Unity after the neural network has completed processing the image and is ready to send the next movement command, but, because the neural networks complete in stochastic time, an additional topic per environment would likely need to be created for the training algorithm to alert the camera script in Unity3D that it is allowed to publish for this effect. This is not similar to how the camera hardware works. Because of this, the training algorithm does not subscribe to the camera's image topic. It receives only the newest image published to the topic when it is ready to receive the image at that point in the training step.

For batching, a parallel for-loop calls a function that undergoes one step of the training function, each parallel execution corresponding to a different simulated batch. Issues arose here such as all the camera publishers taking data from the same camera object, rather than the camera object for the environment. This issue had to be caught by observing all robots making very similar actions as learning proceeded, despite the random distributions causing them to be in different locations when making those movement, as well as copying the images being processed during training into a folder for manual checking.

V. CONCLUSIONS

There are many differences between PPO and Hybrid A* pathfinding aside from their efficacy in pathfinding that bring nuance to their comparison. However, the Hybrid A* algorithm was able to produce a shorter path and traverse that faster than the PPO algorithm was able to on average, in the 28.68% of cases where the Dingo was able to successfully complete paths generated by the trained PPO algorithm. Hybrid A* had the advantage of starting with complete information of the space and therefore not needing to collect information and gain an understanding over time in a long training process. The main downside of Hybrid A* remains to be the runtime seconds needed for creating the mapping and calculating the path.

The PPO algorithm took nearly a week of training time, achieved a low success rate, and did not consistently produce or traverse paths comparably to the Hybrid A* algorithm. This training time did not include the lengths of time spent testing different reward functions through trial and error, which took a significant amount of time. Each time the reward function was trained, the neural network was fully retrained. Different reward functions had different tuning errors that would only be found after many hours of training and heavy debugging to see if the reward trended upwards as the Dingo progressed between waypoints. The results suggest that the Hybrid A* algorithm is superior for this use case of traversing a single desired path in a fully mapped and static space.

A. Future Work

Only one machine learning algorithm and one classical pathfinding approach were evaluated in this research. Future work could evaluate other pathfinding approaches, such as other machine learning approaches such as DQN [13], modified classical utilizing machine learning [8, 12], metaheuristic optimization approaches such as Ant Colony Optimization (ACO) [4], or other classical approaches such as D* Lite. These approaches have been evaluated in the literature before but rarely against one another, as they are typically compared homogeneously – against other approaches in their category – rather than heterogeneously.

Another direction for future work would be a comparison of these pathfinding algorithms in the physical Dingo. This paper involved comparisons in simulation, and, though the simulation was a digital twin made to mimic reality, the experimentation did not establish whether the transfer of this model into the physical Dingo robot would maintain performance in simulation. Research involving the difficulties of transfer learning and the hazards of real-world machine learning training would be valuable future work.

REFERENCES

- [1] Guidance Automation. *A Guide to Autonomous Mobile Robots (AMRs) and 8 AMR Applications*. <https://www.guidanceautomation.com/a-guide-to-autonomous-mobile-robots-amrs-and-8-amr-applications/>. Accessed: 2024-03-12. 2024.
- [2] Nikhil Barhate. *Minimal PyTorch Implementation of Proximal Policy Optimization*. <https://github.com/nikhilbarhate99/PPO-PyTorch>. 2021.
- [3] Dmitri Dolgov et al. “Practical search techniques in path planning for autonomous driving”. In: *ann arbor* 1001.48105 (2008), pp. 18–80.
- [4] Marco Dorigo, Mauro Birattari, and Thomas Stutzle. “Ant colony optimization”. In: *IEEE computational intelligence magazine* 1.4 (2007), pp. 28–39.
- [5] Junli Gao et al. “Deep reinforcement learning for indoor mobile robot path planning”. In: *Sensors* 20.19 (2020), p. 5493.
- [6] AM Hafiz. “A Survey of Deep Q-Networks Used for Reinforcement Learning: State of the Art”. In: *Intelligent Communication Technologies and Virtual Mobile Networks: Proceedings of ICICV 2022* (2022), pp. 393–402.
- [7] Peter E Hart, Nils J Nilsson, and Bertram Raphael. “A formal basis for the heuristic determination of minimum cost paths”. In: *IEEE transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107.
- [8] Ariel Keselman et al. “Reinforcement Learning with A* and a Deep Heuristic”. In: *arXiv preprint arXiv:1811.07745* (2018). arXiv: 1811.07745.
- [9] HoWon Kim and WonChang Lee. “Real-Time path planning through Q-learning’s exploration strategy adjustment”. In: *2021 International conference on electronics, information, and communication (ICEIC)*. IEEE. 2021, pp. 1–3.
- [10] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “Imagenet classification with deep convolutional neural networks”. In: *Advances in neural information processing systems* 25 (2012).
- [11] Yuhanes Liauw et al. “Experimental Comparison of A* and D* Lite Path Planning Algorithms for Differential Drive Automated Guided Vehicle”. In: *Lecture Notes in Electrical Engineering* 282 (Jan. 2014), pp. 555–564. DOI: 10.1007/978-3-642-41968-3-55. (Visited on 12/06/2023).
- [12] Xiaohuan Liu et al. “Novel best path selection approach based on hybrid improved A* algorithm and reinforcement learning”. In: *Applied Intelligence* 51 (2021), pp. 9015–9029.
- [13] Volodymyr Mnih et al. “Playing Atari with Deep Reinforcement Learning”. In: *arXiv preprint arXiv:1312.5602* (2013). arXiv: 1312.5602.
- [14] Erik Nordeus. *Self-driving-vehicle*. <https://github.com/Habrador/Self-driving-vehicle>. 2022.
- [15] Jonathan Platt and Kenneth Ricks. “Comparative analysis of ros-unity3d and ros-gazebo for mobile ground robot simulation”. In: *Journal of Intelligent & Robotic Systems* 106.4 (2022), p. 80.
- [16] Mohamed Reda et al. “Path Planning Algorithms in the Autonomous Driving System: A* Comprehensive Review”. In: *Robotics and Autonomous Systems* 174 (2024), p. 104630. ISSN: 0921-8890. DOI: 10.1016/j.robot.2024.104630.
- [17] Unity Robotics. *ROS-TCP-Endpoint*. <https://github.com/Unity-Technologies/ROS-TCP-Endpoint>. 2022.
- [18] John Schulman et al. “High-dimensional continuous control using generalized advantage estimation”. In: *arXiv preprint arXiv:1506.02438* (2015).
- [19] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [20] Anthony Stentz et al. “The Focussed D* Algorithm for Real-Time Replanning”. In: *IJCAI*. Vol. 95. 1995, pp. 1652–1659.
- [21] Qingfeng Zhang et al. “Path planning of mobile robot in dynamic obstacle avoidance environment based on deep reinforcement learning”. In: *IEEE Access* (2024).
- [22] Junwu Zhao, Ting Qu, and Fang Xu. “A deep reinforcement learning approach for autonomous highway driving”. In: *IFAC-PapersOnLine* 53.5 (2020), pp. 542–546.
- [23] Wei Zhao et al. “Intelligent Vehicle Path Planning Based on Q-learning Algorithm with Consideration of Smoothness”. In: *2020 Chinese Automation Congress (CAC)*. 2020, pp. 4192–4197. DOI: 10.1109/CAC51589.2020.9326831.